

Optimal Buying Strategies in Decentralized Market:

Staking Based Local Identity

Radosław Tereszczuk

Contents

Notation	2
1 Staking Based Identity	2
1.1 Context and Executive Summary	2
2 The Model	3
2.1 Summary	3
2.2 Contract Functions	3
3 The Transaction	3
3.1 Buyer Perspective: Seller Stake	3
3.2 Seller Perspective: Buyer Stake	4
4 Details	4
4.1 Non-Local Verification Models	5
4.2 SWOT Analysis	6
5 Stake Roles	6
5.1 Practical Point of View	6
6 Alternative Cost: Stake Amount Analysis	7
6.1 Buyer Perspective Analysis	7
6.1.1 Rational Evil Seller Rule	7
6.1.2 Buyer Modification of the Rational Evil Seller Rule	8
6.1.3 Rational Buyer Rule	9
6.1.4 Reasonable Buyer Validity Cost Check	10
6.1.5 Reasonable Buyer Result Check Ratio	10
7 Conclusion	11
7.1 Simple Answer	11
8 Seller Perspective Analysis	11
8.1 Conclusion	12

Notation

Throughout this paper the following shorthand notation is used in displayed equations to keep formulae within the page width. All full names are used in the running text.

Symbol	Full name
RCR	Result check ratio
ESR	Evil seller ratio
$Scam_r$	Scam ratio
JV	Job value
N_{buy}	Number of buyers
N_{sell}	Number of sellers
CR	Compound rate
$Stake_{AC}$	Stake alternative cost
$Stake_{amt}$	Stake amount

1 Staking Based Identity

1.1 Context and Executive Summary

We analyze the decentralized market usage scenarios where some amount of tokens is stored in a so-called *stake*, which can be described as a “lock account”. The “lock account” has a property of time delay between a pay-back request and actual execution of the money-returning transfer.

We analyze staking-based use cases from different angles to have as much insight as possible into the basic question:

What should be a stake amount that, for reasonable actors in the network, will make scam non-profitable?

The above question has two versions:

- Being a buyer, what is the minimum stake the seller I would like to cooperate with should have? (we call this perspective the “Buyer perspective analysis”)
- Being a seller, what is the minimum stake the buyer I would like to cooperate with should have? (we call this perspective the “Seller perspective analysis”)

We finish with the following results:

- From the buyer perspective, defining minimal $SellerStake_{amount}$, the seller should stake to make seller-side evil actions not profitable.
- From the buyer perspective, not knowing Job value (JV), the minimal reasonable Result check ratio (RCR).
- From the buyer perspective, the minimal reasonable Validity check cost.

- From the seller perspective, assuming unlimited buyers in the network and $RCR = 1$, defining minimal $BuyerStake_{amount}$ that the buyer should stake to make buyer-side evil actions not profitable.

Besides strict equations, we provide some insights on how the market stake equilibrium will act under the influence of other market variables.

2 The Model

2.1 Summary

Network payment tokens are used for staking by Seller / Buyer to create “stake-based trust”, reducing the probability of Sybil attacks. Stake-based trust can be used to assess the credibility of both potential Sellers and Buyers in the decentralized market market.

In our reasoning we assume that the payment is expected to be executed after delivering the results. Batch payment variants are not analysed.

2.2 Contract Functions

- `payable stake` — transfer network payment tokens and lock them.
 - If for the given originating address the `lockDuration` is not defined, the `defaultLockDuration` value is used (being some short period — for example one hour).
- `setLockDuration(lockDuration)` — sets lock time for the funds staked for this address.
 - The `lockDuration` is expressed in some abstract ticks amount (can be seconds).
 - The new `lockDuration` might not be lower than the previous `lockDuration` value defined for this address.
 - The defined `lockTime` is always global for a given address, meaning `getStakeInfo(address)` will always return one value for all the stakes transferred into the given address.
- `getStakeInfo(address)` — returns $(amount \times lockDuration)$; no gas needed.
- `unlock(amount)` — amount will be moved from locked to unlocked after a delay.
- `withdraw(amount)` — amount will be withdrawn from unlocked, if possible.

3 The Transaction

3.1 Buyer Perspective: Seller Stake

- The Buyer wants to mitigate the risk of the Seller not doing the requested job and, for example, sending trash as a computation result.

- The Seller wants to prove he is trustworthy by putting $SellerStake_{amount}$ into the stake contract.
- The Buyer might perform optional result verification. If the verification gives a negative result, this Buyer will perhaps not execute any future transaction with the given Seller.

Where:

- The $SellerStake_{amount}$ is defined as:

$$SellerStake_{amount} = SellerStake_{value} \times SellerStake_{lock\ time}$$

- The Seller can withdraw stake at any moment, but until all the funds in the stake return to the Seller, $SellerStake_{lock\ duration}$ delay is performed (the lock).
- If the Seller had not set its own value of $SellerStake_{lock\ time}$ through `setLockDuration(lockDuration)` call, the `defaultLockDuration` would be used for $SellerStake_{lock\ time}$.

3.2 Seller Perspective: Buyer Stake

- The Seller wants to mitigate the risk of the Buyer not paying for tasks performed by the Seller.
- The Buyer wants to prove he is trustworthy by putting $BuyerStake_{amount}$ into the stake contract.
- The Seller might perform optional monitoring of the payment and if there is no proper payment for the transaction with a given Buyer, the Seller will perhaps not execute any future transactions with the given Buyer.

Where:

- The $BuyerStake_{amount}$ is defined as:

$$BuyerStake_{amount} = BuyerStake_{value} \times BuyerStake_{lock\ duration}$$

- The Buyer can withdraw stake at any moment, but until all the funds in the stake return to the Buyer, $BuyerStake_{lock\ duration}$ delay is performed (the lock).
- If the Buyer had not set its own value of $BuyerStake_{lock\ time}$ through `setLockDuration(lockDuration)` call, the `defaultLockDuration` would be used for $BuyerStake_{lock\ time}$.

4 Details

- Staking-based reputation as a framework is not relying on any concrete form of computation results verification. This makes the solution a bit simpler.
- The fact of performing Buyer-side verification is unknown to the Sellers. In the reasoning below, we use the Result check ratio (RCR), which is the probability that the results will be checked.

- The Buyer can perform no verification at all and just use the *SellerStake_{amount}* when choosing which Seller to use for a transaction.
- Staking-based reputation to be useful requires some “bad experience” (or perhaps “good experience” also) node list being stored locally in the Buyer / Seller nodes. This historic information will be used for future interactions.
- To have an opinionated history, the node needs to perform some kind of verification. The form of that verification is under sole node control. Without such verification the “bad experience” node list will always be empty.
- What is important: the form and frequency of such verification is up to the actors in the network.
- Depending on the role (Buyer / Seller) the verification can be:
 - Computation verification (verification done by Buyer)
 - Proper payment execution (verification done by Seller)

4.1 Non-Local Verification Models

- If the verification is local (based only on own history), the reasonable required stake amounts are significantly higher than in shared reputation models.
- This is due to the fact that in local verification models, the expected knowledge about an external actor is significantly lower compared to shared reputation models. As the expected knowledge is lower in uncertain conditions, a reasonable assumption is that an unknown actor is evil.
- Any form of reasonable non-local verification model requires a mechanism that can be used to identify false history information being fed into the non-local reputation pool. Such identification is needed to punish the false information author.
- There are two levels of public information required for false history verification:
 - Task input/output hashes being available for public read. This option forms a minimal base for future investigation (assuming full input/output information can potentially be available in future for public read, thus allowing hashes to be compared).
 - Full input/output data, possibly encrypted. Reasonable implementation probably requires SGX/TEE.
- The question about the incentive for non-local reputation pool contribution, especially stake-based, has no known answer.

4.2 SWOT Analysis

Strengths	Weaknesses
Utility scenario for network payment tokens. Easy to understand. Simple to implement. Good anti-Sybil attack defence. The stakes are not generated for each transaction on the market, so if there are many transactions, the gas cost of creating a stake is negligible.	Result verification required by the Buyer (no built-in network support for it). The Sybil attack can still happen — it is just more expensive.
Opportunities	Threats
Urgently needed utility for network payment tokens connected with decentralized computing.	Non-local reputation schema might be a better choice. For the buyer, we need some form of verification mechanism: low-cost verify-all-results ($RCR = 1$) or higher-cost random result check ($RCR < 1$).

5 Stake Roles

There are two main ways of thinking about Stake:

1. **Alternative cost generator.** This is especially valid if the evil actor has unlimited amounts of coins that can be used for unlimited stake creation. Then the alternative (Compound-like) cost is the only limit.
2. **Coin assets eater.** When an evil actor has limited funds (especially in the range needed for a planned attack), the alternative cost is no longer the limiter. The sole amount of funds limits the possible Sybil attack, especially its scale.

Unfortunately, we can assume that any number of coins can be borrowed for an hour or two, possibly creating opportunities for average evil attackers to do possibly a lot of harm if we do not define a reasonable minimum unlock time.

5.1 Practical Point of View

From the practical point of view, we can assume that:

- There are always limits in the amount of tokens available.
- Borrowing large amounts of coins for longer periods of time might not be as easy as borrowing for a few hours.
- Borrowing costs can play a serious role, as there might be some more attractive alternative for the lender.

Take away: we need to make sure that it is more financially attractive to do fund-based evil actions somewhere else.

Turning the above into practical advice: there always has to be some minimal $Stake_{lock\ time}$ long enough to create a financially attractive alternative option for a potential lender, so that the evil actor would find borrowing funds for evil action on the decentralized market unattractive.

When we construct the exact staking scheme we do not have to decide how we exactly think about stake (“alternative cost generator” or “coin assets eater”) as both narratives are based on similar models that use:

- $Stake_{value}$
- $Stake_{lock\ time}$

We just need some $Stake_{minimal\ lock\ time}$ built into the stake model, potentially to be defined/suggested by the network.

6 Alternative Cost: Stake Amount Analysis

At some point, we need a precise definition of the stake amount versus other factors. These analyses are performed from the buyer and seller perspectives.

For simplicity, we assume each job has the same computation complexity and price (Job value, JV).

For the stake amount we use the alternative cost perspective:

$$Stake_{amt} = Stake_{value} \times Stake_{lock\ time}$$

$$Stake_{AC} = Stake_{amt} \times CR$$

Where:

- $Stake_{lock\ time}$ is the lock time during `unlock(amount)`
- CR (Compound rate) is an example alternative cost generator

6.1 Buyer Perspective Analysis

The buyer must define some minimal $Stake_{amt}$ that will be defined in his demand that the seller needs to have staked.

6.1.1 Rational Evil Seller Rule

An evil seller does not want the stake-based identity creation to have a cost higher than expected income from the scam:

$$Stake_{AC} < Scam_{income(N)}$$

Remarks:

- We omit income from proper execution of tasks.

- Other costs are intentionally omitted.

Where $Scam_{income(N)}$ is the expected value of one evil seller income after N jobs:

$$Scam_{income(N)} = Scam_r \times (1 - RCR) \times JV \times N$$

Where:

- $Scam_r = \frac{N_{scam\ jobs}}{N_{all\ jobs}}$
- RCR (Result check ratio) — the probability that the result of a given job will be locally tested for validity.

6.1.2 Buyer Modification of the Rational Evil Seller Rule

The rational evil seller rule from above is now modified with the buyer perspective. The different perspective changes the direction of the inequality sign:

$$Stake_{AC} > Scam_{income(N)}$$

As a buyer, I want the seller to have his $Stake_{AC}$ higher than $Scam_{income(N)}$.

We want to find some boundary $Stake_{AC}$ where:

$$Stake_{AC} = Scam_{income(N)} \tag{1}$$

Assuming that one evil seller can try to scam everyone ($Scam_r = 1$) of all the buyers in the network, the expected value of $Scam_{income(N)}$:

When $N \leq N_{buy}$:

$$Scam_{income(N)} = (1 - RCR) \times N \times JV$$

When $N > N_{buy}$:

$$Scam_{income(N)} = (1 - RCR) \times JV \times \left(N_{buy} + \frac{1 - (1 - RCR)^{N - N_{buy}}}{RCR} \right)$$

Where N_{buy} is the number of buyers in the network.

As an evil seller would like to maximize his $Scam_{income(N)}$, he will want N to be highest. But as the $N > N_{buy}$ part of $Scam_{income(N)}$ has exponential nature (and the base of the power is below 1), the gains are lower and lower for $N > N_{buy}$. In the limit (for $N \rightarrow \infty$):

$$\lim_{N \rightarrow \infty} Scam_{income(N)} = (1 - RCR) \times JV \times \left(N_{buy} + \frac{1}{RCR} \right)$$

So as the evil seller would like to maximize his $Scam_{income(N)}$, the boundary $Stake_{AC}$ is:

$$Stake_{AC} = (1 - RCR) \times JV \times \left(N_{buy} + \frac{1}{RCR} \right) \tag{2}$$

As a buyer I want the seller to have his stake to be at least:

$$Stake_{amt} = \frac{(1 - RCR) \times JV \times \left(N_{buy} + \frac{1}{RCR} \right)}{CR}$$

Of course, assuming $RCR = 1$, there could be no stake at all.

6.1.3 Rational Buyer Rule

This is a not-strictly-provable assumption, but let's assume that a rational buyer wants his cost generated by task result verification to be less than the expected scam loss:

$$Result\ check\ cost(N) < Scam_{loss(N)} \quad (3)$$

Where $Scam_{loss(N)}$ is the expected value of one buyer loss after N jobs.

$$\begin{aligned} Result\ check\ cost(N) &= RCR \times N \times Validity\ check\ cost \\ Scam_{loss(N)} &= JV \times Scam_N(N) \end{aligned}$$

Where:

- RCR — probability that the result of a given job will be locally tested for validity.
- $Validity\ check\ cost$ — cost of checking if one job result is valid.
- $Scam_N(N)$ — expected number of scams after N jobs.

Now let's estimate $Scam_N(N)$. Let $\alpha = RCR \times Scam_r$ for brevity:

$$Scam_N(N) = RCR \times Scam_r \times ESR \times \frac{1 - ESR^{N/\alpha}}{1 - ESR}$$

Where:

- ESR (Evil seller ratio) — probability that a given seller in the network is an evil one.

Remarks. The above unknown sellers interaction model is a very simple one. It assumes that:

- In the beginning there is zero knowledge of evil sellers.
- In the next steps the knowledge about evil sellers is not used.
- If a non-evil seller is doing its job, we can use it for any number of future jobs.

As the real ESR and $Scam_r$ are never known, we can use a simple model, as even a proper model will not give us full details because of the unknowns.

In the end we have the following form of the Rational Buyer Rule (with $\alpha = RCR \times Scam_r$):

$$\begin{aligned} \text{Result check cost}(N) &< JV \times RCR \times Scam_r \times ESR \\ &\times \frac{1 - ESR^{N/\alpha}}{1 - ESR} \end{aligned}$$

6.1.4 Reasonable Buyer Validity Cost Check

By using equation (3) and dividing both sides by $RCR \times N$:

$$\begin{aligned} \text{Validity check cost} &< JV \times Scam_r \times ESR \\ &\times \frac{1}{N} \cdot \frac{1 - ESR^{N/\alpha}}{1 - ESR} \end{aligned}$$

6.1.5 Reasonable Buyer Result Check Ratio

Now we will try to express the minimal reasonable Result check ratio (RCR) in terms of other parameters of the model. It is possible to be expressed without knowing the Job value JV .

We start with:

$$\text{total scam loss} = \text{total scam income}$$

In our case:

$$Scam_{income(N)} \times ESR \times N_{sell} = N_{buy} \times Scam_{loss(N)}$$

With the use of equations (1) and (2) we can say that:

$$JV \times \left((1 - RCR) \times N_{buy} + \frac{1 - RCR}{RCR} \right) = Scam_{income(N)}$$

As the evil seller wants to maximize his income, $Scam_{loss}$ converges:

$$Scam_{loss} = \lim_{N \rightarrow \infty} Scam_{loss(N)} = JV \times RCR \times Scam_r \times \frac{ESR}{1 - ESR}$$

In the end we have:

$$\begin{aligned} &\left((1 - RCR) \times N_{buy} + \frac{1 - RCR}{RCR} \right) \times ESR \times N_{sell} \\ &= N_{buy} \times RCR \times Scam_r \times \frac{ESR}{1 - ESR} \end{aligned}$$

We can reduce JV , and after symbolic solving we have the quadratic equation with coefficients (using $n_r = N_{buy}$, $n_p = N_{sell}$, $e_r = ESR$, $s = Scam_r$):

$$\begin{aligned}
a &= n_r \cdot s + (n_r - n_r \cdot e_r) \cdot n_p \\
b &= ((n_r - 1) \cdot e_r - n_r + 1) \cdot n_p \\
\Delta &= (4n_r - 4n_r e_r) n_p s \\
&\quad + ((n_r^2 + 2n_r + 1) e_r^2 \\
&\quad + (-2n_r^2 - 4n_r - 2) e_r + n_r^2 + 2n_r + 1) n_p^2
\end{aligned}$$

The Result check ratio RCR is one of the results that has a positive value:

$$RCR = \frac{-b - \sqrt{\Delta}}{2a} \quad \text{or} \quad RCR = \frac{-b + \sqrt{\Delta}}{2a}$$

7 Conclusion

1. As ESR (Evil seller ratio) and $Scam_r$ (Scam ratio) are always unknown, we cannot have precisely defined $Stake_{amt}$ requirements.
2. In a real market, there are many factors that will form the market equilibrium on the $Stake_{amt}$. Some of them are:

Bigger seller stake required	Smaller seller stake required
Higher ESR [non-lin]	Lower ESR [non-lin]
Lower RCR [lin]	Higher RCR [lin]
Lower CR [lin]	Higher CR [lin]
Higher Validity check cost [lin]	Lower Validity check cost [lin]
Higher JV [lin]	Lower JV [lin]
Lower N_{buy} [lin]	Higher N_{buy} [lin]
Higher N_{sell} [lin]	Lower N_{sell} [lin]

7.1 Simple Answer

A good rule of thumb (although not guaranteeing anything) is:

$$Stake_{AC} > JV$$

So:

$$Stake_{amt} > \frac{JV}{CR}$$

8 Seller Perspective Analysis

The seller must define some minimal $Stake_{amt}$ that will be defined in his offer that the buyer needs to have staked.

Assuming unlimited buyers in the network and $RCR = 1$, and as the evil buyer in the worst case scenario performs a scam on every seller in the network:

$$Stake_{AC} > N_{sell} \times JV$$

That means:

$$Stake_{amt} = \frac{N_{sell} \times JV}{CR}$$

8.1 Conclusion

As the number of buyers is limited, the $Stake_{amt}$ will be defined by market equilibrium. Some factors:

Bigger buyer stake required	Smaller buyer stake required
Lower CR [lin]	Higher CR [lin]
Higher cost [lin]	Lower cost [lin]
Higher N_{buy} [lin]	Lower N_{buy} [lin]
Lower N_{sell} [lin]	Higher N_{sell} [lin]